

swe645-hw2

Peter Shin (G01073633)

What I Built

A static student survey web form for "Totally Real University", containerized with Docker, deployed on a Kubernetes cluster managed via Rancher on AWS EC2, and continuously delivered via a Jenkins CI/CD pipeline.

Live deployment: <http://54.175.211.106:30610/>

Docker image: <https://hub.docker.com/r/frozenmandu/swe645-hw2>

Architecture

- `index.html` — Bootstrap 5 dark-themed single-page form with no backend; submission is client-side only
- `Dockerfile` — Ubuntu 24.04 base, installs nginx, serves `index.html` on port 80
- `Jenkinsfile` — CI/CD pipeline: Checkout → Build → Push → Deploy, plus a post-build cleanup step
- `kubernetes/deployment.yaml` — Kubernetes Deployment with 3 replicas
- `kubernetes/service.yaml` — NodePort Service exposing the app externally

How It Works

Runtime (serving the app): The static `index.html` page is containerized on an Ubuntu base image running nginx with port 80 exposed. That image is published to a Docker Hub registry. On the Kubernetes cluster, a Deployment pulls the image and runs **3 replica pods** for resiliency — if a pod dies, Kubernetes automatically recreates it to maintain 3. A NodePort Service fronts those pods and exposes the app externally on port **30610**, so the site is reachable at <http://<EC2-public-IP>:30610/>.

CI/CD (shipping changes): Jenkins automates the pipeline end to end. On each push to `main`, a GitHub webhook triggers the Jenkins job, which: (1) checks out the repository from GitHub, (2) builds the Docker image and tags it with the Jenkins build number, (3) pushes that image to Docker Hub, and (4) runs `kubectl set image` to point the Deployment at the new image — Kubernetes then performs a rolling update so the new version goes live with no downtime. The net effect: a `git push` automatically ships the change to the running site.

Prerequisites

Before starting, you'll need:

- An AWS EC2 instance running Ubuntu (this host runs Rancher, the Kubernetes node, and Jenkins)
- A Docker Hub account with a personal access token (PAT)
- A GitHub account (this repo)

All tools (Docker, kubectl, Rancher, Jenkins) are installed on the EC2 host during setup below.

Setup & Installation

Run these steps **in order** on a fresh Ubuntu EC2 instance (SSH in first). By the end, the cluster is running and the Jenkins pipeline auto-deploys the app on every push.

1. Configure the EC2 Security Group

Allow the following inbound ports on the instance's Security Group:

Port	Purpose
22	SSH
80, 443	Rancher UI (HTTP/HTTPS)
8080	Jenkins web UI
30000–32767	Kubernetes NodePort range (the app is published on 30610)

2. Install Docker on the EC2 host

```
sudo su -  
apt-get update && apt upgrade -y  
apt install -y docker.io  
systemctl start docker  
systemctl enable docker
```

3. Run Rancher and create the Kubernetes cluster

Start the Rancher container:

```
docker run --privileged -d --restart=unless-stopped -p 80:80 -p 443:443  
rancher/rancher  
docker ps    # record the Rancher container ID
```

Then create the cluster in the Rancher UI:

1. Open <https://<EC2-public-IPv4-DNS>> in a browser (may take a minute to come up).
2. Follow the on-screen instructions to retrieve the bootstrap password, then set a new password.
3. Create a new cluster → choose **Custom** → name it → **Create**.
4. Under **Step 1**, select all three node roles: **etcd**, **Control Plane**, **Worker**.
5. Under **Step 2**, set TLS to **Insecure**, then copy the generated registration command and run it on the same EC2 host. The node registers and the cluster becomes active.

4. Connect kubectl to the cluster

```
snap install kubectl --classic
mkdir -p ~/.kube
touch ~/.kube/config
```

In the Rancher UI, open the cluster from the dashboard and click **Download Kubeconfig** (top-right). Paste its contents into `~/.kube/config` on the EC2 host, then verify:

```
kubectl get nodes
```

5. Install Jenkins

Jenkins runs on the same EC2 instance and listens on port **8080**. These steps follow the official [Jenkins Debian/Ubuntu install guide](#).

Install Java (Jenkins requires a JDK/JRE):

```
sudo apt update
sudo apt install fontconfig openjdk-21-jre
java -version
```

Install Jenkins (LTS release):

```
sudo wget -O /etc/apt/keyrings/jenkins-keyring.asc \
https://pkg.jenkins.io/debian-stable/jenkins.io-2026.key
echo "deb [signed-by=/etc/apt/keyrings/jenkins-keyring.asc] \
https://pkg.jenkins.io/debian-stable binary/ | sudo tee \
/etc/apt/sources.list.d/jenkins.list > /dev/null" | sudo tee \
/etc/apt/sources.list.d/jenkins.list
sudo apt update
sudo apt install jenkins
```

Start Jenkins and confirm it's running:

```
sudo systemctl start jenkins
sudo systemctl status jenkins
```

Unlock and complete first-run setup:

- Open `http://<EC2-public-IP>:8080` in a browser.
- Retrieve the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

- Paste it, choose **Install suggested plugins**, then create the admin user and finish.

6. Configure Jenkins for the pipeline

1. Install plugins (if not already present): Git, Docker Pipeline, Workspace Cleanup, Pipeline Stage View.
2. Add credentials under **Manage Jenkins → Credentials**:
 - **ID:** `dockerhub-creds` — Kind: Username with password (Docker Hub username + PAT)
 - **ID:** `kubeconfig-id` — Kind: Secret file (your cluster's kubeconfig)
3. Give Jenkins access to Docker: `sudo usermod -aG docker jenkins` (restart Jenkins afterward).
4. Create a Pipeline job pointing to this repo's `Jenkinsfile`.
5. Configure a GitHub webhook pointing to `http://<EC2-public-IP>:8080/github-webhook/` so pushes trigger builds.

7. Deploy the application

The Jenkins pipeline deploys automatically, but you can also create the Deployment and Service manually the first time:

```
kubectl apply -f kubernetes/deployment.yaml
kubectl apply -f kubernetes/service.yaml
```

Verify and access the app:

```
kubectl get pods
kubectl get service hw2
```

The service is type `NodePort` — access the app at `http://<EC2-public-IP>:<NodePort>`. For this deployment that is `http://54.175.211.106:30610/` (NodePort `30610` is pinned in `kubernetes/service.yaml`).

CI/CD Pipeline

The Jenkins pipeline triggers automatically on every push to `main` via the GitHub webhook configured in setup step 6.

Pipeline Stages

Stage	Description
Checkout	Clones the repo from GitHub
Build	Builds the Docker image tagged with <code>\$BUILD_NUMBER</code> using the Docker Pipeline plugin

Stage	Description
Push to Docker Hub	Pushes the image to Docker Hub using the <code>dockerhub-creds</code> Jenkins credential
Deploy	Runs <code>kubectl set image</code> to update the Kubernetes deployment in-place (skippable via <code>DEPLOY</code> parameter)

A `post { always }` step runs after every build to remove the local Docker image from the EC2 instance and prevent disk buildup.

Local Development

Optional — build and run the image directly with the Docker CLI, without Kubernetes or Jenkins.

Build and run locally:

```
docker build -t frozenmandu/swe645-hw2:latest .
docker run -p 8081:80 frozenmandu/swe645-hw2:latest
```

Then open `http://localhost:8081` in your browser.

Build and push to Docker Hub:

```
docker login -u <your-dockerhub-username>
docker build -t frozenmandu/swe645-hw2:<tag> .
docker push frozenmandu/swe645-hw2:<tag>
```